PUCP

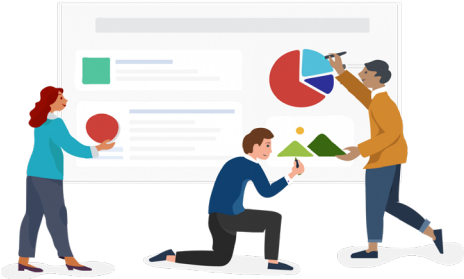
PROGRAMACIÓN 3 (1INF30)

Librerías y Componentes Reutilizables en C#

2026

Profesores:

- Dr. Freddy Paz
- Dr. José Luis Corcuera
- Mag. Eric Huiza
- Mag. David Allasi



AGENDA

1. ¿Qué es una DLL?
2. Librería VS DLL
3. Tipos de Proyectos en .NET
4. Aplicación de Consola
5. Creación de una Aplicación de Consola
6. Espacios de nombres – Namespaces
7. Namespace y Using en C#
8. Modificadores de acceso en clases en C#
9. Diferencia con JAVA
10. Organización del Código en C#
11. Proyecto de Biblioteca de Clases en C#
12. Creación de un Proyecto de Biblioteca de Clases – C#
13. Consumo de una Biblioteca de Clases – Mediante Referencia al Proyecto / Referencia Manual

¿Qué es una DLL?

- DLL significa **Dynamic Link Library**, y representa una biblioteca utilizada en aplicaciones .NET.
- Una DLL es un archivo compilado que contiene clases, métodos y otros elementos de código reutilizable.
- Una DLL puede ser utilizada por múltiples aplicaciones, lo que permite compartir funcionalidad y evitar duplicación de código.

Librería VS DLL

- Una librería es un concepto que representa código reutilizable dentro de un sistema.
- Una DLL es la forma física en la que una librería se distribuye en .NET.
- En este sentido, toda DLL es una librería, pero una librería no es necesariamente un archivo.

Tipos de Proyectos en .NET

- En .NET existen distintos tipos de proyectos que permiten desarrollar aplicaciones ejecutables o componentes reutilizables.
- Las **aplicaciones de consola** y **aplicaciones web** funcionan como programas completos que pueden ser ejecutados por el usuario o un servidor.
- Las **librerías de clases** generan archivos DLL que encapsulan funcionalidades reutilizables dentro del sistema.
- Todos estos proyectos se compilan en ensamblados que contienen el código intermedio ejecutado por la plataforma .NET.

Aplicación de Consola

- Una **aplicación de consola** genera un ensamblado principal en formato DLL que contiene el código compilado del programa. Adicionalmente, se genera un archivo ejecutable (.exe) que actúa como lanzador de la aplicación.
- En aplicaciones .NET modernas, el código compilado se encuentra dentro del archivo DLL. **El archivo .exe no contiene la lógica del programa, sino que se encarga de iniciar la ejecución.** Esto permite que el mismo código pueda ejecutarse en diferentes plataformas utilizando el **runtime** de .NET.

Creación de una Aplicación de Consola

- Una aplicación de consola puede ser creada utilizando la herramienta de línea de comandos de .NET.
- El comando `dotnet new console` permite generar automáticamente la estructura base del proyecto.
- Este proyecto incluye un archivo principal (`Program.cs`) donde se define el punto de entrada del programa.
- Posteriormente, el comando `dotnet build` compila el código y genera los ensamblados necesarios para su ejecución.
- Finalmente, el comando `dotnet run` permite ejecutar la aplicación desde la consola.

Creación de una Aplicación de Consola

- El comando `dotnet new console` permite crear una aplicación de consola con un nombre por defecto.
- El parámetro `-n` permite especificar el nombre del proyecto de forma explícita.
- Es recomendable utilizar `-n` para mantener un mejor control sobre la organización de los proyectos.

```
C:\Windows\System32\cmd.e

Microsoft Windows [Versión 10.0.26200.8117]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Prog3>dotnet new console -n Principales
La plantilla "Aplicación de consola" se creó correctamente.

Procesando acciones posteriores a la creación...
Restaurando C:\Prog3\Principales\Principales.csproj:
Restauración realizada correctamente.
```

Creación de una Aplicación de Consola

- Al crear una aplicación de consola, **.NET** genera automáticamente una estructura básica de proyecto.
- El archivo `Program.cs` contiene el punto de entrada donde se inicia la ejecución del programa.
- El archivo `.csproj` define la configuración del proyecto, incluyendo dependencias y versión del **framework**.
- La carpeta `bin/` contiene los archivos compilados, como el ensamblado en formato **DLL** y el archivo ejecutable.
- La carpeta `obj/` almacena archivos temporales utilizados durante el proceso de compilación.

Espacios de nombres - Namespaces

- Un **espacio de nombres** es un mecanismo que permite organizar las clases y otros elementos dentro de un proyecto.
- Su función principal es facilitar la estructuración del código.
- En C#, los espacios de nombres son conceptualmente equivalentes a los **paquetes** en Java. Sin embargo, a diferencia de los paquetes, los espacios de nombres no dependen obligatoriamente de la estructura de carpetas del proyecto. En C#, el **namespace** es una organización lógica, mientras que en Java el **package** suele estar ligado a la estructura física del proyecto

Namespace y Using en C#

- La palabra clave `namespace` permite definir el espacio de nombres al que pertenece una clase.
- La palabra clave `using` permite acceder a los elementos de un espacio de nombres sin necesidad de escribir su nombre completo.
- Esto facilita la escritura del código y mejora su legibilidad.

Namespace y Using en C#

Operacion.cs

```
1  namespace Matematicas{
2      class Operacion{
3          public int Sumar(int a, int b){
4              return a + b;
5          }
6      }
7  }
```

Principal.cs

```
1  namespace Principales{
2  using Matematicas;
3      class Principal{
4          public static void Main(){
5              Operacion op = new Operacion();
6              Console.WriteLine(op.Sumar(10,20));
7          }
8      }
9  }
```

Namespace y Using en C#

- El **namespace** organiza el código, y el **using** evita tener que escribir rutas largas.
- También es posible no utilizar la palabra clave **using** y acceder a una clase escribiendo su ruta completa, incluyendo el espacio de nombres, aunque esto puede hacer el código más extenso y menos legible.

Namespace y Using en C#

Operacion.cs

```
1  namespace Matematicas{
2      class Operacion{
3          public int Sumar(int a, int b){
4              return a + b;
5          }
6      }
7  }
```

Principal.cs

```
1  namespace Principales{
2      class Principal{
3          public static void Main(){
4              Matematicas.Operacion op = new Matematicas.Operacion();
5              Console.WriteLine(op.Sumar(10,20));
6          }
7      }
8  }
```

¿Por qué la clase `Operacion` es accesible desde otro espacio de nombres a pesar de no ser pública?

- En C#, una clase que no tiene modificador de acceso se define por defecto como `internal`.
- El modificador `internal` permite que la clase sea accesible dentro del mismo ensamblado.
- Esto significa que cualquier clase dentro del mismo proyecto puede utilizarla sin necesidad de que sea pública.
- Sin embargo, la clase no podrá ser utilizada desde otros ensamblados o proyectos externos.

Modificadores de acceso en clases en C#

- En **C#**, las clases pueden definirse con distintos modificadores de acceso que determinan su visibilidad.
- Por defecto, una clase se define como `internal`, lo que significa que solo es accesible dentro del mismo ensamblado.
- El modificador `public` permite que la clase sea accesible desde cualquier proyecto que la referencie.
- El modificador `internal` restringe el acceso únicamente a las clases que se encuentran dentro del mismo proyecto.

Acceso para Clases en C#

Modificador	Mismo Proyecto	Otro Proyecto
internal	✓	✗
public	✓	✓

Diferencia con JAVA

- En **Java**, el acceso de las clases está determinado por el **paquete**, mientras que en **C#** está determinado por el **ensamblado**.
- En Java, las clases pueden definirse como **public** o sin modificador de acceso.
- Una clase **public** puede ser utilizada desde cualquier **paquete** del sistema.
- Una clase sin modificador se considera de acceso por defecto (**default**), y solo es accesible dentro del mismo paquete.

Organización del Código en C#

- En C#, los espacios de nombres representan una organización lógica del código y no dependen directamente de la estructura física de carpetas.
- Sin embargo, **es posible crear carpetas** dentro del proyecto para organizar mejor los archivos.
- Esta organización física facilita la comprensión, el mantenimiento y la escalabilidad del sistema.
- Aunque no es obligatorio mantener coherencia entre carpetas y espacios de nombres, es una buena práctica.

Proyecto de Biblioteca de Clases en C#

- Un proyecto de biblioteca de clases permite crear código reutilizable que puede ser utilizado por múltiples aplicaciones.
- Este tipo de proyecto no genera un programa ejecutable, sino un ensamblado en formato DLL.
- La DLL contiene clases, métodos y otros componentes que encapsulan la lógica del sistema.
- Estas bibliotecas pueden ser referenciadas desde otros proyectos para compartir funcionalidad.

Proyecto de Biblioteca de Clases en C#

- Una biblioteca de clases puede ser creada utilizando la herramienta de línea de comandos de .NET.
- El comando **dotnet new classlib -n MiLibreria** permite generar un proyecto de tipo librería.
- Este proyecto está diseñado para compilarse en un ensamblado en formato DLL.
- El comando **dotnet build** compila el proyecto y genera el archivo DLL con el código reutilizable.
- Esta DLL puede ser referenciada desde otras aplicaciones para reutilizar su funcionalidad.

Creación de un Proyecto de Biblioteca de Clases – C#

C:\Windows\System32\cmd.e

```
C:\Prog3>dotnet new classlib -n Matematicas
La plantilla "Biblioteca de clases" se creó correctamente.

Procesando acciones posteriores a la creación...
Restaurando C:\Prog3\Matematicas\Matematicas.csproj:
Restauración realizada correctamente.
```

Consumo de una Biblioteca de Clases en .NET

- Una biblioteca de clases puede ser utilizada por otra aplicación mediante la incorporación de su ensamblado o mediante una referencia directa al proyecto.
- Existen dos formas principales de consumir una biblioteca de clases en .NET: **mediante una referencia al proyecto**, cuando se dispone del código fuente o la biblioteca ha sido desarrollada por nosotros, y **mediante un proceso manual** utilizando directamente el archivo DLL.

Consumo de una Biblioteca de Clases en .NET

- Una biblioteca de clases puede ser utilizada por otra aplicación mediante la incorporación de su ensamblado o mediante una referencia directa al proyecto.
- Existen dos formas principales de consumir una biblioteca de clases en .NET: **mediante una referencia al proyecto**, cuando se dispone del código fuente o la biblioteca ha sido desarrollada por nosotros, y **mediante un proceso manual** utilizando directamente el archivo DLL.

Mediante una Referencia al Proyecto

- Este enfoque se emplea cuando se dispone del código fuente de la biblioteca o cuando ha sido desarrollada dentro de la misma solución.
- La referencia se realiza utilizando el comando **dotnet add reference**, indicando la ruta del proyecto de biblioteca.
- Una vez establecida la referencia, la aplicación puede utilizar las clases públicas de la biblioteca de forma directa.
- Además, cualquier cambio en la biblioteca se refleja automáticamente al recompilar la aplicación.

Mediante una Referencia al Proyecto

```
C:\Windows\System32\cmd.e

C:\Prog3\Principales>dotnet add reference ../Matematicas/Matematicas.csproj
Se ha agregado la referencia "..\Matematicas\Matematicas.csproj" al proyecto.

C:\Prog3\Principales>
```

```
C:\Windows\System32\cmd.e

C:\Prog3\Principales>dotnet build
Restauración completada (0.5s)
  Matematicas net10.0 realizado correctamente (0.2s) → C:\Prog3\Matematicas\bin\Debug\net10.0\Matematicas.dll
  Principales net10.0 realizado correctamente (0.4s) → bin\Debug\net10.0\Principales.dll

Compilación realizado correctamente en 1.6s

C:\Prog3\Principales>
```

Mediante una Referencia al Proyecto

- De forma automática, se genera en el archivo **.csproj**

Principales.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <ItemGroup>
    <ProjectReference Include="..\Matematicas\Matematicas.csproj" />
  </ItemGroup>

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

Mediante una Referencia Manual

- El consumo manual consiste en utilizar directamente el archivo DLL.
- Este enfoque se emplea cuando no se dispone del código fuente o cuando la biblioteca es distribuida como archivo compilado.
- Para utilizar la biblioteca, es necesario copiar o ubicar el archivo DLL en una ruta accesible para la aplicación.
- Posteriormente, la aplicación debe agregar una referencia al archivo DLL para poder utilizar sus clases.

Mediante una Referencia Manual

- Se modifica el archivo **.csproj**, considerando en este ejemplo que el archivo DLL se encuentra dentro de una carpeta "**libs**" en el directorio del proyecto "**Principales**"

Principales.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <Reference Include="Matematicas">
      <HintPath>libs\Matematicas.dll</HintPath>
    </Reference>
  </ItemGroup>

</Project>
```

Distribución de una librería

- Una o más librerías se pueden empaquetar y distribuir a través del sistema de gestión de paquetes distribuibles de .NET llamado **NuGet**.
- El paquete distribuible se genera usando el comando dotnet pack. El archivo generado tiene la extensión .nupkg.

dotnet pack /p:PackageId="Matematicas" -o ../paquetes

Nota: -o especifica el folder donde se almacenará el paquete

Este comando debe ejecutarse en la carpeta donde está el archivo .csproj del proyecto de biblioteca de clase.

Referenciar un paquete NuGet

- Para agregar una referencia a un paquete NuGet se usa el comando dotnet package add.
- Este comando referenciará las DLLs incluídas dentro del paquete, en el proyecto.
- Este método se usa cuando las librerías están listas para ser distribuidas entre equipos y compañias.

dotnet package add Matematicas -s ../paquetes

Nota: -s especifica el folder donde se encuentra el paquete.

Referencias

- D.J. Barnes y M. Kölling, Programación orientada a objetos con Java. Pearson Educación, 2007
- P. Deitel y Harvey, Visual C# 2012 How to Program, 2013.
- J. Sharp, Microsoft Visual C# 2013 Step by Step. Microsoft Press, 2013
- T. Budd, An introduction to Object-Oriented Programming (Third Edition). Pearson Education, 2001
- E. Gamma, R. Helm, R. Johnson, J. Vlissides, Design Paterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994
- Agustín Froufe. Java 2. Manual de usuario y tutorial. Ed. Ra-Ma
- J. Sánchez, G. Huecas, B. Fernández y P. Moreno, Iniciación y referencia: Java 2. Osborne McGraw-Hill, 2001.

¡Gracias!

PUCP